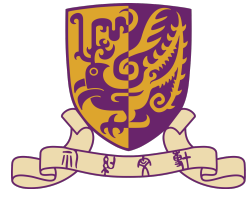
CSCI3230 (ESTR3108)
Fundamentals of Artificial Intelligence

Tutorial. Neural Network Basics

Siyuan Huang

Email: syhuang26@cse.cuhk.edu.hk
Office: Room 120, 1/F, SHB

Dept. of Computer Science & Engineering
The Chinese University of Hong Kong



Outline

Part 1. Neural network basics

Part 2. Backpropagation

Part 3. Coding with PyTorch

By the end of this tutorial

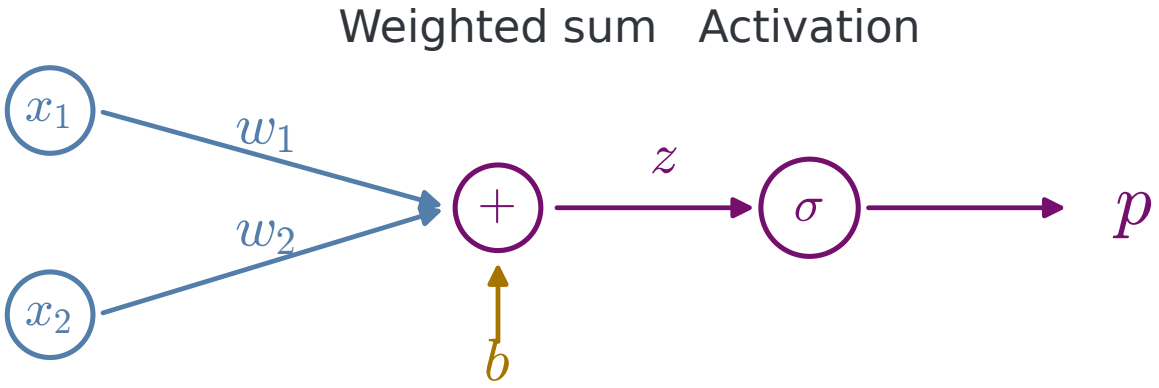
Trace a forward pass, check gradients, and train a small network.



Part 1. Neural network basics

One output neuron for binary classification

$$z = w_1 x_1 + w_2 x_2 + b, \qquad p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

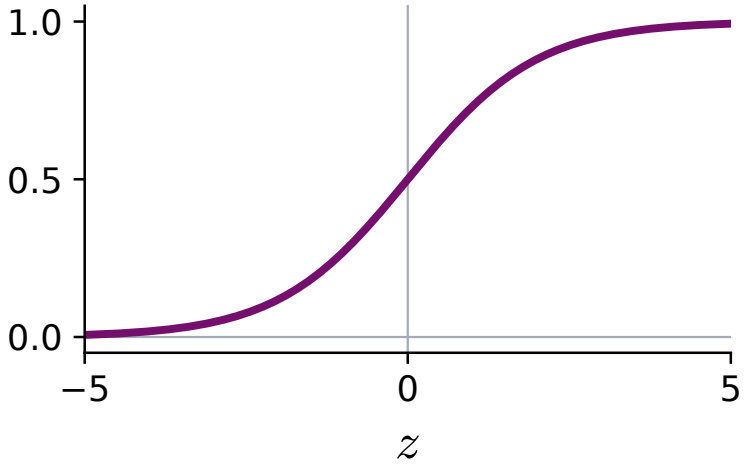


- x_1, x_2: input features. We learn the weights w_1, w_2 and bias b.
- z is a **logit**: a score that can be any real number.
- Sigmoid maps z to $p \in (0, 1)$, our estimate of $P(y = 1 \mid x)$.

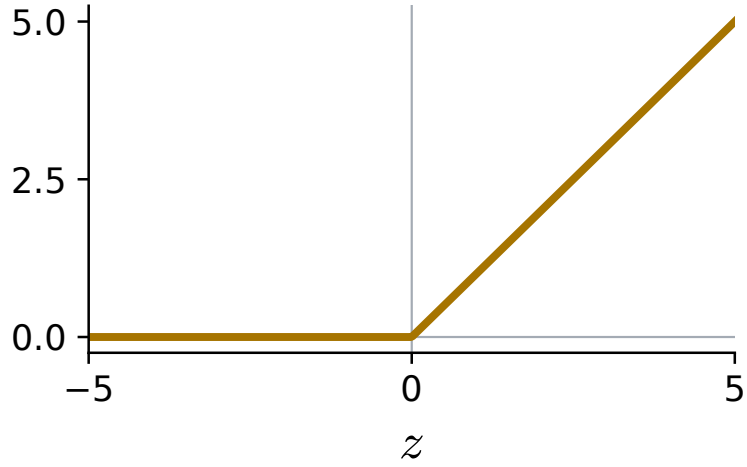
For example, $z = 0$ gives $p = 0.5$; $z = 2$ gives $p \approx 0.88$.

Two activation functions we will use

Sigmoid: output probability



ReLU: hidden activation



$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \mathrm{ReLU}(z) = \max(0, z).$$

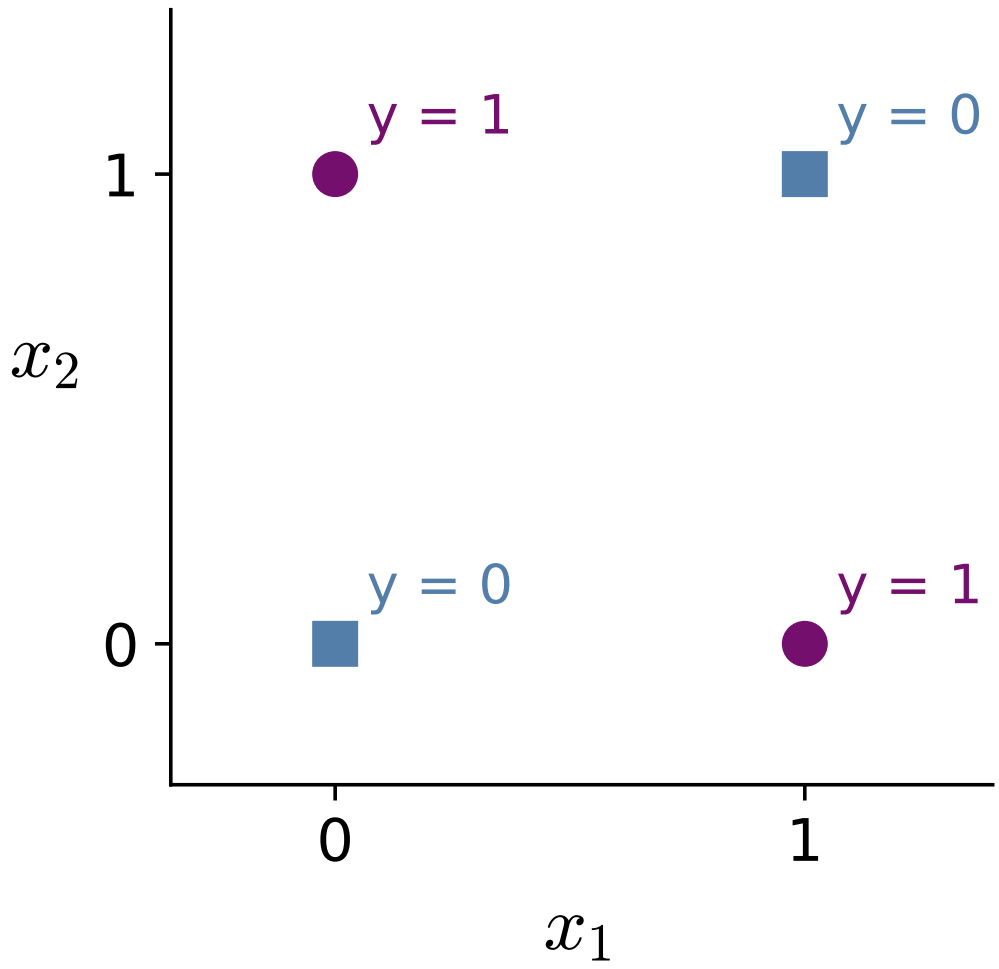
- ReLU sets negative inputs to zero and keeps positive inputs.
- We apply ReLU to each hidden value to add nonlinearity.
- Sigmoid maps the final logit to a probability between 0 and 1.

Our task: learn XOR

Output 1 when the two inputs differ.

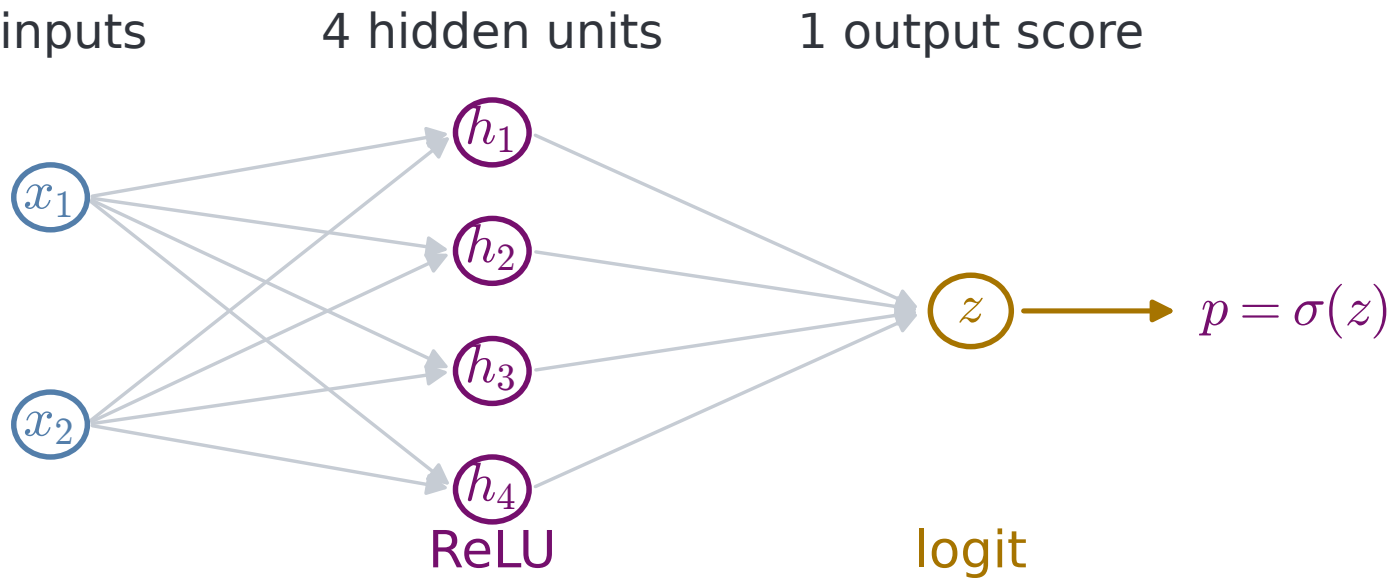
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Can one straight line separate the two classes?



One straight decision boundary cannot separate these four points.

A small feedforward network



$$h = \mathrm{ReLU}(W_1 x + b_1), \qquad z = W_2 h + b_2.$$

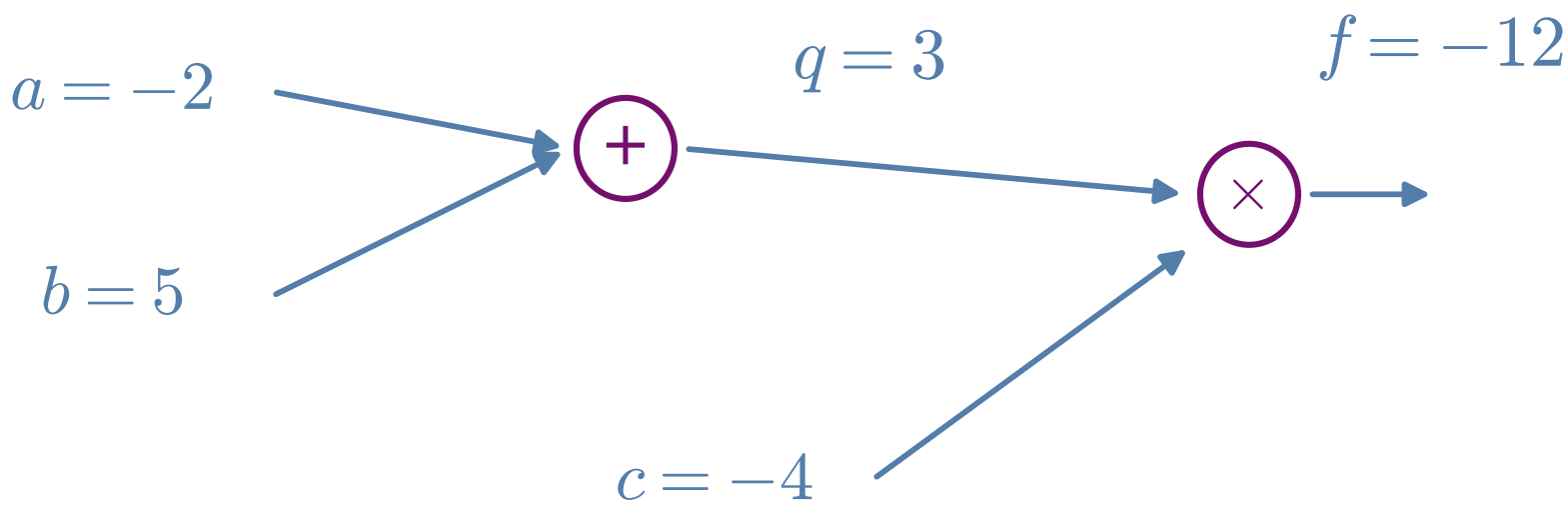
- x has 2 features; h contains the 4 hidden neurons' outputs.
- Each hidden neuron forms its own weighted sum, then applies ReLU.
- The output layer combines these 4 values into z; sigmoid gives p.



Part 2. Backpropagation

A computation graph from the lecture

$$f = (a + b)c, \qquad a = -2, \quad b = 5, \quad c = -4.$$

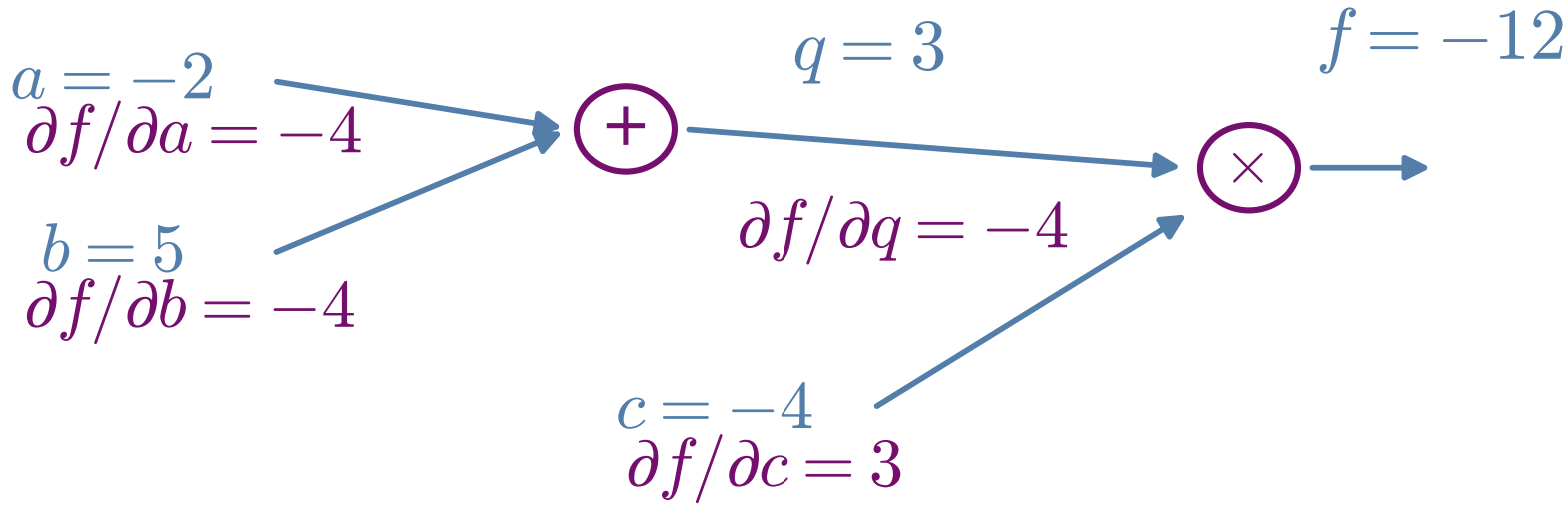


Forward pass

First compute $q = a + b = 3$. Then compute $f = qc = -12$.

Next, find how much f changes when we change one input.

Backward pass: follow the chain rule



At the multiplication node, $\partial f/\partial q = c = -4$.

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial a} = (-4)(1) = -4.$$

Similarly, $\partial f/\partial b = -4$. For the other input, $\partial f/\partial c = q = 3$.

Check the same gradients with PyTorch

```
import torch
```

```
a = torch.tensor(-2.0, requires_grad=True)
b = torch.tensor(5.0, requires_grad=True)
c = torch.tensor(-4.0, requires_grad=True)
f = (a + b) * c
f.backward()
print(f.item())
print(a.grad.item(), b.grad.item(), c.grad.item())
```

Expected output

```
-12.0
-4.0  -4.0  3.0
```

`requires_grad=True` enables tracking. `backward()` computes gradients; `.grad` stores them.

From gradients to learning

For our binary task, let $p = \sigma(z)$ and $y \in \{0, 1\}$.

Binary cross-entropy for one sample

$$\ell = -\big[y \log p + (1 - y) \log(1 - p)\big].$$

- $y = 1$: $\ell = -\log p$. $\quad y = 0$: $\ell = -\log(1 - p)$.
- Giving the correct class a higher probability lowers the loss.
- Backpropagation finds how each weight and bias affects the loss.

$$\theta_{\text{new}} = \theta - \eta \frac{\partial L}{\partial \theta}.$$

L: mean loss over our four samples. $\quad \eta$: learning rate.

θ: any weight or bias. Subtract its gradient to reduce the loss locally.



Part 3. Coding with PyTorch

Coding: data and model

Four training samples:

```
X = torch.tensor([
    [0., 0.], [0., 1.],
    [1., 0.], [1., 1.]
])
y = torch.tensor([
    [0.], [1.], [1.], [0.]
])
```

Each row of `X` is a sample.
Each row of `y` is its label.

Our $2 \to 4 \to 1$ network:

```
torch.manual_seed(42)
model = nn.Sequential(
    nn.Linear(2, 4),
    nn.ReLU(),
    nn.Linear(4, 1)
)
```

`Linear(2,4)`: 2 inputs $\to$ 4 sums.
`ReLU()`: apply to each sum.
`Linear(4,1)`: 4 values $\to$ 1 logit.

Both `nn.Linear` layers include a bias.
Setup: `import torch`, `from torch import nn`.
The fixed seed makes this demonstration reproducible.

Check shapes before training

Quantity	Shape	Meaning
`X`	$(4, 2)$	4 samples, 2 features
`y`	$(4, 1)$	1 label per sample
`model(X)`	$(4, 1)$	1 logit per sample
Hidden activations	$(4, 4)$	4 values per sample

```
print(X.shape, y.shape, model(X).shape)
print(model[0].weight.shape, model[2].weight.shape)
print(sum(p.numel() for p in model.parameters()))
```

PyTorch stores a linear layer's weights as (output features, input features).

How many trainable parameters?

Hidden: 4×2 weights $+ 4$ biases.
Output: 1×4 weights $+ 1$ bias. Total: 17.

Choose the loss and the optimizer

```
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
```

- `BCEWithLogitsLoss` combines sigmoid and binary cross-entropy.
- Pass logits directly: do not apply sigmoid before this loss.
- Its default reduction is the mean over our four samples.
- SGD updates all trainable parameters with learning rate 0.1.

Two different jobs

Backpropagation computes the gradients.
The optimizer uses them to update the parameters.

The training loop

```
losses = []
for epoch in range(3000):
    optimizer.zero_grad()
    logits = model(X)
    loss = loss_fn(logits, y)
    loss.backward()
    optimizer.step()
    losses.append(loss.item())
```

- `zero_grad()`: clear old gradients; PyTorch adds new ones to them.
- `model(X)`: forward pass; `loss_fn(logits,y)`: compare with labels.
- `backward()`: compute gradients; `step()`: update parameters.

Each step uses all four samples; `losses` records the training loss.

Turn logits into predictions

```
with torch.no_grad():
    probabilities = torch.sigmoid(model(X))
    predictions = (probabilities >= 0.5).float()
print(probabilities)
print(predictions)
```

$$p = \sigma(z), \qquad \hat{y} = \begin{cases} 1, & p \geq 0.5, \\ 0, & p < 0.5. \end{cases}$$

- p: probability estimate. $\hat{y}$: predicted class, 0 or 1.
- `no_grad()` disables gradient tracking during prediction.
- This cell uses the trained parameters; it does not update them.

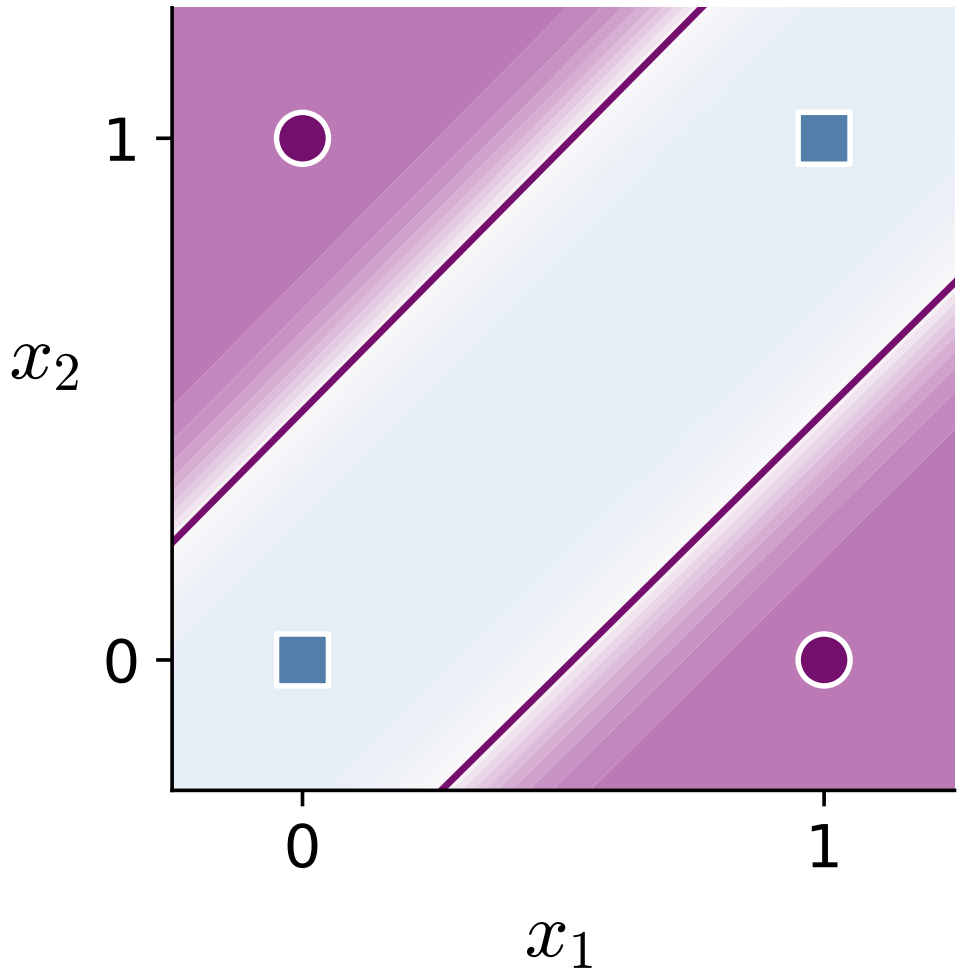
What did the network learn?

After 3000 training steps:

x_1	x_2	y	p	$\hat{y}$
0	0	0	0.005	0
0	1	1	0.997	1
1	0	1	0.993	1
1	1	0	0.004	0

Mean training loss: 0.0046.

All four training points are classified correctly.



Purple curve: $p = 0.5$.
Squares: class 0. Circles: class 1.

Compare ReLU with Identity

`nn.Identity()` replaces the hidden ReLU activation.

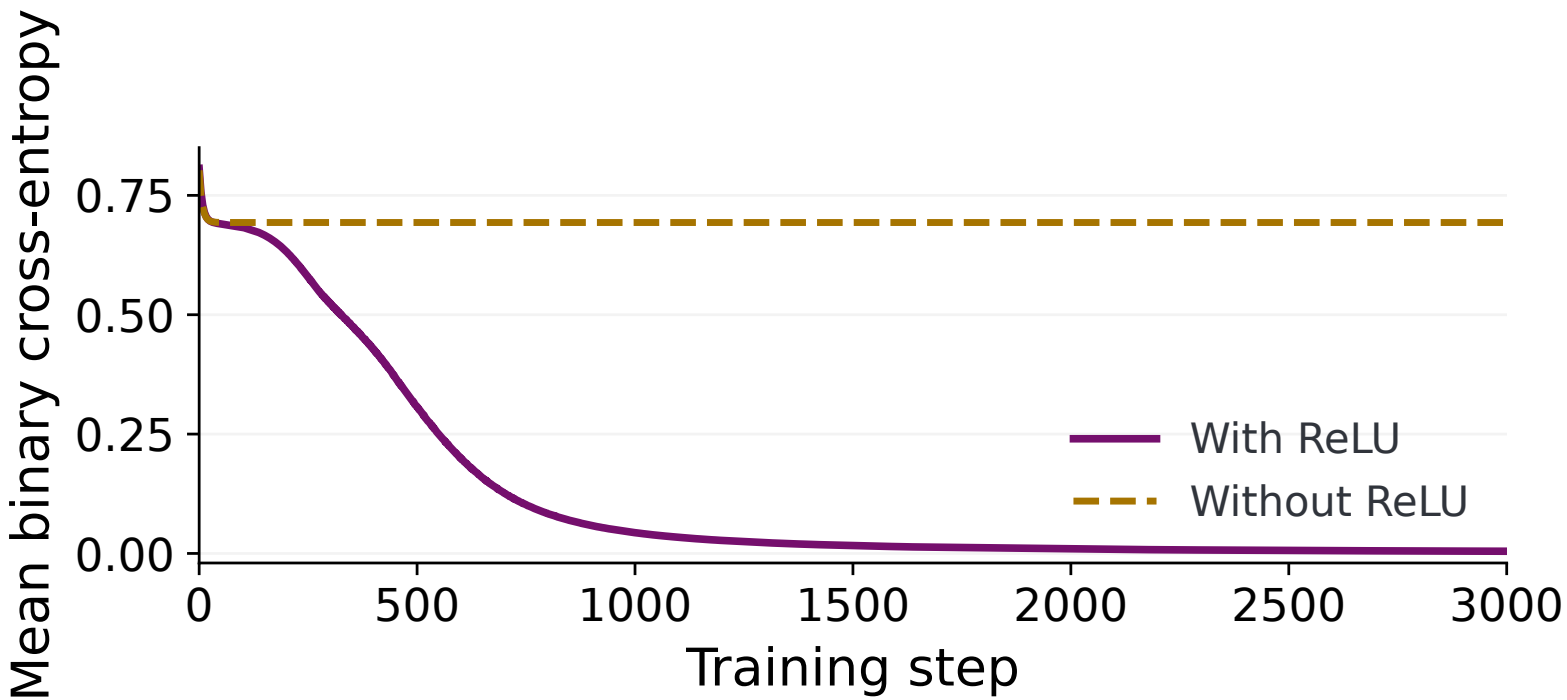
```
activation = nn.Identity()
experiment_model, experiment_losses = run_experiment(
    activation
)
```

- Identity returns its input unchanged.
- The `run_experiment` function creates a fresh model and optimizer.
- Both runs use the same seed, data, layer sizes and training settings.
- We compare the final loss, probabilities and training curves.

Why removing ReLU changes the model

Without ReLU, the output becomes

$$z = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$



The decision boundary is straight, so the model cannot separate XOR.
In this run, the loss is 0.6931 and all four probabilities are about 0.5.

Take-away

- A forward pass computes the network's output.
- Nonlinear activations let hidden layers learn richer functions.
- Backpropagation applies the chain rule to compute gradients.
- The training loop clears gradients, computes the loss, backpropagates, and updates the parameters.

When you work with a real dataset

Check tensor shapes and monitor the training loss.
Evaluate predictions on held-out data.

Further reading: PyTorch autograd and optimization tutorial.